



Все курсы

Тота ИИ



Шаг пройден

[🏠](#) / [Мое](#) / [Мои проекты](#) / AI-агент службы поддержки

Проект: AI-агент службы поддержки

15 августа
2026 г.

> НЕДОСТ...

📄 НЕДОСТ...

○ В ПРОЦ...

[🔗 Репозиторий](#)[🔗 Помощь](#)

Подготовка

✓ 1

✓ 2

> 3

> 4

> 5

> 6

> 7

> 8

> 9

> 10

[📄 Проверка](#)

Настройка Yandex AI Studio + права и секреты

Настраиваем доступы: создаём сервисный аккаунт, выдаём ему минимально необходимые роли (включая `ydb.editor`) и помещаем параметры подключения и токены в Yandex Lockbox — чтобы нигде в коде и в `.env` не хранились секреты в открытом виде. Это фундамент: без правильной авторизации следующие шаги (MCP, workflow) не заработают — появится ошибка `Service account required`.

Главный принцип — никаких секретов в коде. В проекте три категории секретов:

- Параметры подключения к YDB (`YDB_ENDPOINT`, `YDB_DATABASE`) — для Cloud Function `ydb-tickets`.
- API-ключ AI Studio — для обращения к моделям.
- App-password для почтового ящика — для авторизации по IMAP/SMTP; его добавим в Lockbox на шаге про email-workflow.

Все они живут в Lockbox. Cloud Function читает секреты через `yc-lockbox` интеграцию (`--secret environment-variable=secret-id/version-id/key`), а не из `env`.

Ссылки

- [Как начать работать с Managed Service for YDB](#)



Все курсы

1. Создайте сервисный аккаунт `ai-studio-sa`.

```
1 yc iam service-account create --name ai-studio-sa
```



2. Назначьте роли в каталоге. Команда `add-access-binding` принимает только одну роль за вызов — поэтому повторяем её для каждой роли. Явно передайте `--id <folder-id>` (или `--name help-desk`), иначе `yc` упадёт с `either --id, --name or positional arg are required`:

```
1 SA_ID=$(yc iam service-account get --name ai-studio-sa --format json | jq -r .id)
2 FOLDER_ID=$(yc config get folder-id)
3 for ROLE in \
4     functions.functionInvoker \
5     serverless.mcpGateways.invoker \
6     lockbox.payloadViewer \
7     ai.languageModels.user \
8     ydb.editor; do
9     yc resource-manager folder add-access-binding \
10         --id "$FOLDER_ID" \
11         --service-account-id "$SA_ID" \
12         --role "$ROLE"
13 done
```

Проверка: `yc resource-manager folder list-access-bindings --id "$FOLDER_ID" | grep ai-studio-sa` — должны увидеть все 5 ролей. Роль `ydb.editor` нужна, чтобы Cloud Function `ydb-tickets` (шаг про MCP и YDB) могла делать `insert` и `select` в YDB от имени SA. Роль `ai.languageModels.user` нужна, чтобы вызывать модели через Responses API (шаг про `email-workflow`) — без неё вызовы вернут 403. Альтернатива — `set-access-bindings` за один вызов, но она стирает существующие `binding`'и каталога; используйте только в чистом каталоге.

3. Создайте секреты в Lockbox:

- `ydb-endpoint` → значение вида `grpc://...:2135` (из шага подготовки).
- `ydb-database` → значение вида `/ru-central1/b1g.../etn...` (из шага подготовки).
- `ai-studio-api-key` (опционально) → ключ из `https://aistudio.yandex.ru` → Настройки → API-ключи. Нужен, только если планируете вызывать модели напрямую через SDK снаружи Yandex Cloud (шаг про QA). Для Cloud Function внутри YC можно обойтись SA + `metadata service` — тогда этот секрет не нужен.

Шаблон команды (один секрет — один вызов):



Все курсы

```
3 --payload ' [{"key": "value", "text_value": "grpc://ydb.serverless.yandexcloud.net:2135
```

Ключ `payload` (`value` , `token` , и т.п.) — на ваш выбор; он же потом пойдёт в `--secret VAR=id/ver/key` при деплое Cloud Function.

4. Включите AI Studio в каталоге (через консоль — однократно) при первом обращении к <https://aistudio.yandex.ru> .

5. Создайте агента `help-desk` в UI Agent Atelier:

- Имя: `help-desk` , модель: `yandexgpt` (или `yandexgpt-lite` для дешёвых прогонов).
- System prompt: составьте его сами. Промпт должен задавать роль Help Desk-агента и тон, предписывать краткость и определять поведение «если ответа не знаю — честно сказать и предложить создать тикет». Промпт расширится на шагах про MCP и про RAG.
- Сохраните агента → из URL скопируйте `agent_id` (строка вида `fvtv6oos1ous46c9squf`).

SA к агенту не привязывается — в текущей версии AI Studio этой опции в UI нет. Авторизация работает по-другому (см. ниже «Как работает авторизация»).

Как работает авторизация

1. SA прикрепляется к Cloud Function, которая вызывает AI Studio Responses API (функция `email-poller` на шаге про `email-workflow`). Флаг `--service-account-id` в `yc serverless function version create` .
2. Cloud Function получает свежий IAM-токен через `metadata service` YC (`http://169.254.169.254/computeMetadata/v1/instance/service-accounts/default/token`) — не нужно вручную обновлять токен.
3. AI Studio выполняет шаги агента от имени IAM-токена вызывающего — то есть от имени `ai-studio-sa` .
4. Когда агент вызывает MCP-tool (шаг про MCP и YDB), MCP-сервер проверяет у IAM-токена роль `serverless.mcpGateways.invoker` на каталог. Эта роль уже выдана в задаче 2.

Альтернатива для публичных MCP-серверов — переключить MCP-tool в режим `authorization: none` в UI MCP Hub. Тогда вызов идёт без проверки IAM, но и без логирования «кто вызывал». Для учебного проекта рекомендуется приватный MCP + SA на Cloud Function.

Источники: <https://aistudio.yandex.ru/docs/ru/ai-studio/concepts/mcp-hub/> (раздел «General MCP server settings»).



Все курсы

- `YDB_ENDPOINT` указывайте со схемой `grpc://` — без нее Cloud Function не подключится к базе.
- Проверить, что все роли у SA на месте:

```

1  yc resource-manager folder list-access-bindings --id "$FOLDER_ID" --format json \
2  | jq -r '.[ ] | .role_id as $r | .subject | select(.type=="serviceAccount") | "\($r)\t(.i
3  | while IFS=$'\t' read -r role_id; do
4      name=$(yc iam service-account get "$id" --format json | jq -r .name)
5      printf '%s\t%s\t%s\n' "$role" "$name" "$id"
6  done

```

Результат шага

- 2 секрета в Lockbox (`ydb-endpoint` , `ydb-database` ; `ai-studio-api-key` опционален при схеме SA+metadata; `app-password` для почты добавится на шаге про email-workflow): ID'ы записаны в `.env` (НЕ значения).
- Сервисный аккаунт `ai-studio-sa` создан с ролями `functions.functionInvoker` , `serverless.mcpGateways.invoker` , `lockbox.payloadViewer` , `ai.languageModels.user` , `ydb.editor` .
- Агент `help-desk` создан в UI Agent Atelier, `agent_id` зафиксирован. SA к агенту НЕ привязывается (см. «Как работает авторизация»).
- В `README.md` проекта зафиксировано: какой секрет куда положен, какие роли у SA.

Завершить шаг

B
I
H1
99
</>
📄
🔗
🔗
🖼️
☰
📦
🔗
👁️

Отправить

Комментарии